Does the Model Know Why? A Failure-Mode Cross-Checking Protocol for Verifying SHAP Explanations in Multi-Mechanism Predictive Maintenance

Koshik Debanath
Independent Researcher
koshik.debanath@gmail.com

Abstract

Post-hoc explanation methods such as SHAP are routinely applied to predictive-maintenance classifiers, but the resulting attributions are rarely checked against anything beyond the analyst's intuition. We use a property of the AI4I 2020 predictive maintenance dataset that is usually discarded: its binary `Machine failure` label is the logical OR of five independently defined failure mechanisms (tool wear, heat dissipation, power, overstrain, and a random failure with no physical cause), each with an exact, published triggering rule. We train an imbalance-aware gradient-boosted classifier on the composite label only – withholding the five sub-labels from the feature set entirely – and then use the sub-labels purely as a post-hoc audit: for each mechanism, we measure the share of the model's mean absolute SHAP attribution, on the rows where that mechanism fired, that falls on the features the mechanism's trigger depends on, and test it against a model-randomization null built from models retrained on permuted labels. We show that a naive null (random subsets of failure rows under the same model) is inadequate: a model trained on permuted labels passes it, because mechanism rows are atypical in their own features. Under the model-randomization null, our tuned XGBoost model (precision 1.00, recall 0.82, F1 0.90 on the held-out failure class) passes all four physical mechanisms ($p \leq 0.02$), while a label-permuted model fails all four and a model given a spurious shortcut sensor fails exactly the mechanism the shortcut mimics – despite that model's accuracy staying competitive. The result holds on rows where exactly one mechanism fired, and three of four mechanisms remain clearly recoverable from raw sensor features alone. We argue this protocol – a dataset's own latent causal substructure used as a faithfulness probe – applies to any predictive-maintenance setting where a coarse target aggregates several distinct, partially-labeled failure modes.

1 Introduction

Predictive maintenance (PdM) models are increasingly built with gradient-boosted trees or neural networks and explained after the fact with SHAP [Lundberg and Lee, 2017]. The explanation is typically presented as evidence that the model "makes sense" – a beeswarm plot, a bar chart of global feature importance – but is rarely checked against an independent source of truth about *why* the underlying process actually failed. This is a meaningful gap: a model can achieve strong discriminative performance while keying off a spurious correlate of failure rather than the mechanism itself, and a plausible-looking SHAP plot will not by itself reveal the difference [Vollert and Theissler, 2024].

The AI4I 2020 dataset [Matzka, 2020] offers an unusual opportunity to close this gap without constructing anything artificial for the purpose. Its target, `Machine failure`, is by design the logical OR of five independent sub-conditions – tool wear failure (TWF), heat dissipation failure

(HDF), power failure (PWF), overstrain failure (OSF), and a random failure (RNF) that fires with fixed probability regardless of process state. Each of the first four has an exact, published closed-form trigger (Table 1); RNF has none. Critically, a model trained to predict `Machine failure` is never given these five labels – it only ever sees the OR of them. This mirrors deployments in which a single "needs maintenance" flag aggregates several distinct root causes that are recorded separately, or not at all, and never reach the model as a training signal.

This gives us a natural faithfulness probe: after training on the composite label alone, we can go back and ask, for the subset of rows where a specific mechanism actually fired, whether the model's SHAP attributions concentrate on the feature that mechanism's own definition implicates. If they do not, that is a warning sign that the classifier is exploiting a different, possibly spurious, correlate. An audit is only meaningful if it can fail, so we also construct models it *should* reject – one trained on permuted labels, one given a spurious shortcut sensor – following the model-randomization logic of Adebayo et al. [2018].

Contributions.

1. An imbalance-aware benchmark of seven model families (Logistic Regression, SVM, k-NN, Decision Tree, Random Forest, XGBoost, and a small PyTorch MLP) on AI4I 2020, with a resampling-technique bake-off (SMOTE vs. Borderline-SMOTE vs. ADASYN), hyperparameter and decision-threshold tuning, and 5-fold cross-validation.

2. A lightweight cross-checking protocol that uses a dataset's own latent, partially-labeled causal substructure – rather than purpose-built synthetic ground truth – to audit whether a trained model's SHAP explanations are faithful to the real generating mechanism, with a model-randomization null. We show the obvious alternative null is confounded and passes a model that learned nothing.

3. Evidence the audit discriminates: the tuned model passes all four physical mechanisms; a label-permuted model fails all four; a model with an injected shortcut fails exactly the mechanism the shortcut mimics while its accuracy stays competitive. We also report an exclusive-mode analysis and a raw-feature ablation addressing circularity.

2 Related Work

Matzka [2020] introduced the AI4I 2020 dataset alongside a LIME-based illustration of local explanations for individual predictions, but did not systematically audit attributions against the five sub-mechanisms across the dataset. Several subsequent works apply SHAP to AI4I 2020 for global feature-importance reporting – e.g., a recent milling-machine study [Zhao et al., 2025] reports SHAP summary plots for tree ensembles trained on this dataset – but, to our knowledge, none of them cross-check per-mechanism attributions against the known triggering rules, nor test whether such a check can fail. A recent physics-informed benchmarking study [Tseng, 2026] constructs engineered features from the same sensor variables and reports a Random Forest positive-class F1 of 0.722 on AI4I 2020, and a connected-vehicles study [Khemani and Qureshi, 2026] benchmarks LightGBM under a stricter SMOTE-confined-to-training-folds cross-validation protocol, reporting macro-F1 0.814 ± 0.012. We compare directly against both in Section 5.

Dhande et al. [2026] take a related but distinct angle: they build formal causal structural models on a similarly-sized CNC failure dataset to optimize maintenance decisions under asymmetric cost, and show causal features generalize better than raw correlational ones. Our goal is narrower and

Table 1: The five failure mechanisms underlying `Machine failure` in AI4I 2020 [Matzka, 2020], with counts as they appear in the released data. Only the composite label is ever shown to the model; the individual flags are withheld from training and used only for the audit in Section 4.5. TWF is only partly determined by the features (the failure point is drawn at random within the window). 18 of the 19 RNF rows have `Machine failure` $= 0$ (see Section 3).

Code	Name	Trigger condition	Count
TWF	Tool wear failure	tool wear falls in a randomly assigned 200–240 min window	46
HDF	Heat dissipation failure	(process temp. − air temp.) < 8.6 K **and** rpm < 1380	115
PWF	Power failure	torque × rpm (rad/s) outside [3500 W, 9000 W]	95
OSF	Overstrain failure	tool wear × torque exceeds a type-specific limit (11/12/13k)	98
RNF	Random failure	fixed 0.1% chance, independent of all process variables	19

complementary – we do not build a causal model of the process; we audit whether an ordinary correlational classifier's *post-hoc explanation* happens to align with the true mechanism, using labels that already encode that mechanism.

Work on faithfulness evaluation for explainability methods generally relies on *synthetic* ground truth: data is generated from a known functional form specifically so that the correct attribution is computable in closed form [Liu et al., 2021], or on randomization tests such as Adebayo et al. [2018], who show that some saliency methods produce near-identical explanations for trained and randomized models. We borrow that randomization logic for our null. Our approach differs in where the ground truth comes from – rather than designing a synthetic task for the purpose of evaluating explanations, we reuse sub-labels that already exist in an otherwise ordinary, publicly available PdM benchmark, aggregated away in the task the model is actually asked to solve. AI4I 2020 is itself a simulated dataset generated from these same closed-form rules, a caveat we return to in Section 6.2.

3 Dataset

AI4I 2020 [Matzka, 2020] contains 10,000 rows and 14 columns: a row identifier and product ID (dropped), a categorical product quality variant (`Type` $\in \{L, M, H\}$, 50/30/20% of rows respectively), five continuous/integer process variables (air temperature, process temperature, rotational speed, torque, tool wear), the composite `Machine failure` label, and the five mechanism indicators of Table 1. `Machine failure` is 1 for 339 of 10,000 rows (3.39%), a severe class imbalance typical of real PdM data.

The released labels are not perfectly consistent with the stated OR rule, and this matters for the audit. 24 rows have more than one mechanism flag set; 9 failures have no mechanism flag at all; and 18 of the 19 RNF rows have `Machine failure` $= 0$. The last point means the RNF subgroup is almost entirely rows the model is supposed to classify as *non*-failures, so it cannot serve as a strong negative control on its own – we still report it, but the discriminating evidence for the audit comes from the positive controls in Section 5.5. We run the audit both on all flagged rows and on

rows where exactly one mechanism fired.

4 Methodology

4.1 Feature engineering

We construct four features directly from the mechanism definitions in Table 1, without using the mechanism labels themselves: *Power* = torque × rpm (rad/s) (mirrors the PWF rule), *Temp_diff* = process temperature − air temperature (mirrors HDF), *Overstrain* = tool wear × torque (mirrors OSF), and *Overstrain_ratio* = Overstrain normalized by the type-specific limit. This gives the model, in principle, direct access to each mechanism's driving quantity without ever seeing which mechanism fired.

4.2 Leakage-safe feature extraction

`UID` and `Product ID` are identifiers and are dropped. TWF, HDF, PWF, OSF, and RNF are components of the target by construction (`Machine failure` $= \bigvee_i \text{mode}_i$) and are excluded entirely from the feature matrix; they are retained only as held-out audit labels.

4.3 Split, resampling, and models

We use a stratified 70/20/10 train/validation/test split. The preprocessor (standard-scaling + one-hot encoding) is fit on the training split only. We compare SMOTE [Chawla et al., 2002], Borderline-SMOTE [Han et al., 2005], and ADASYN [He et al., 2008] by resampling the training split and scoring a fixed default XGBoost [Chen and Guestrin, 2016] on the untouched validation split; the winner (Borderline-SMOTE in our run) is used for the remainder of the pipeline. We train seven model families – Logistic Regression, SVM (RBF), k-NN, Decision Tree, Random Forest, XGBoost, and a small PyTorch feed-forward network – on the resampled training data and evaluate on the untouched, imbalanced validation and test splits.

4.4 Tuning and validation

Model selection uses the validation split only: XGBoost has the highest validation F1 (0.826, against 0.739 for the runner-up, Random Forest) and is the only model tuned further. Tuning uses a 40-iteration, 5-fold stratified randomized search scored on F1. The decision threshold is then chosen on the validation precision-recall curve, maximizing F1, instead of the default 0.5. We additionally run 5-fold stratified cross-validation, refitting preprocessing, resampling, and the tuned hyperparameters inside each fold, for a variance estimate beyond a single 1,000-row test split. Test-set intervals are exact binomial (Clopper–Pearson) for precision and recall and a 5,000-sample stratified bootstrap for F1 and the AUCs.

4.5 XAI cross-checking protocol

This is the core methodological contribution. We compute SHAP `TreeExplainer` values for the final tuned model on all flagged rows (not just the test split – TWF and RNF have under 50 rows in total, too few to analyze within a 1,000-row test partition; this uses no training signal, only post-hoc explanation of an already-fixed model). For mechanism m and feature j,

$$I_{m,j} = \frac{1}{|S_m|} \sum_{i \in S_m} |\phi_j(x_i)| \,,$$

where S_m is the set of rows for which m fired and $\phi_j(x_i)$ is the SHAP value of feature j for row i. Let E_m be the features m's trigger depends on (raw inputs and their derived versions): TWF – tool wear; HDF – air and process temperature, their difference, rotational speed; PWF – torque, rotational speed, power; OSF – tool wear, torque, overstrain, overstrain ratio. The audit statistic is the *relevant share*

$$R_m = \sum_{j \in E_m} I_{m,j} \Big/ \sum_{j} I_{m,j}.$$

Why a naive null fails. The obvious test compares R_m against the same statistic on random, equally sized subsets of failure rows under the same model. This is confounded: rows where m fired are, by definition, extreme in the features of E_m, and $|\phi_j|$ tends to be large at extreme feature values even for a model that learned nothing useful. A model trained on permuted labels passes this naive test on every mechanism (Section 5.5).

Model-randomization null. Following Adebayo et al. [2018], we instead retrain the identical pipeline (same features, resampling, and hyperparameters) $K = 100$ times on permuted training labels and compute R_m for each. A mechanism passes if the real model's R_m exceeds the permuted models' with $p = (1+\#\{R_m^{\text{perm}} \geq R_m\})/(K+1) < 0.05$; the smallest attainable p is $1/101 \approx 0.0099$. For RNF, which has no cause, the statistic is the share of the single top feature, and RNF passes if the real model is *not* more concentrated on RNF rows than the permuted models ($p \geq 0.05$).

5 Results

5.1 Model comparison

Table 2 reports positive-class (`Machine failure`=1) metrics on the held-out test split. XGBoost dominates the comparison even before tuning; tree ensembles substantially outperform the linear and instance-based models, consistent with the class boundary being nonlinear and threshold-based (Section 5.3) rather than a simple linear separation. Hyperparameter and threshold tuning lift positive-class F1 from 0.87 to 0.90, driven almost entirely by a precision gain (0.86 → 1.00) at a moderate recall cost (0.88 → 0.82). These rest on 34 test failures, so the uncertainty is substantial: the tuned model flags 28 rows, all true failures, giving precision 1.00 with 95% CI [0.88, 1.00]; recall is 0.82 [0.65, 0.93], F1 0.90 [0.83, 0.97], and ROC-AUC 0.984 [0.958, 0.999]. Its scores are also well calibrated (Brier score 0.0095 against 0.033 for a constant prevalence prediction; expected calibration error 0.007), although the threshold choice depends only on how scores rank rows.

Table 2: Positive-class (`Machine failure`=1) test-set metrics, sorted by F1.

Model	Precision	Recall	F1	ROC-AUC
XGBoost (tuned)	**1.00**	0.82	**0.90**	0.984
XGBoost (baseline)	0.86	0.88	0.87	0.988
Random Forest	0.79	0.88	0.83	0.989
Decision Tree	0.68	0.82	0.75	0.927
PyTorch MLP	0.53	0.91	0.67	0.978
SVM (RBF)	0.42	0.88	0.57	0.976
k-NN	0.43	0.79	0.56	0.877
Logistic Regression	0.24	0.85	0.37	0.943

Five-fold cross-validation of the tuned model (default 0.5 threshold per fold, hyperparameters fixed) gives F1 $= 0.784 \pm 0.029$, precision $= 0.740 \pm 0.066$, recall $= 0.838 \pm 0.028$ – confirming the single-split test result is not an artifact of a favorable split, and giving a more conservative estimate than the threshold-tuned single-split number in Table 2 (the per-fold threshold was left at 0.5 deliberately, to isolate the model's raw discriminative behavior from the threshold-tuning step).

Table 3: Comparison against prior AI4I 2020 benchmarks. Metric definitions are not identical across studies (macro-F1 averages both classes; positive-class F1 reports the failure class alone, which is substantially harder under 3.39% prevalence) – we report both where available rather than force a single number.

Study	Model	Metric	Value
Tseng [2026]	Random Forest	positive-class F1	0.722
Khemani and Qureshi [2026]	LightGBM (5-fold CV, SMOTE in-fold)	macro-F1	0.814 ± 0.012
This work	XGBoost (tuned, single split)	positive-class F1	**0.90**
This work	XGBoost (tuned, single split)	macro-F1	**0.95**
This work	XGBoost (tuned, 5-fold CV)	positive-class F1	0.784 ± 0.029

5.2 Resampling comparison and decision boundaries

Table 4 shows the validation-set bake-off; Borderline-SMOTE wins on F1 and is used throughout. Figure 2 visualizes the effect directly: a Logistic Regression classifier is fit on a 2D PCA projection of the training data, once per resampler. Without resampling, the decision boundary barely bends toward the minority class; each oversampler pulls it further into majority territory, visibly trading precision for recall – the same tradeoff Table 2 shows numerically for the full feature space and stronger models.

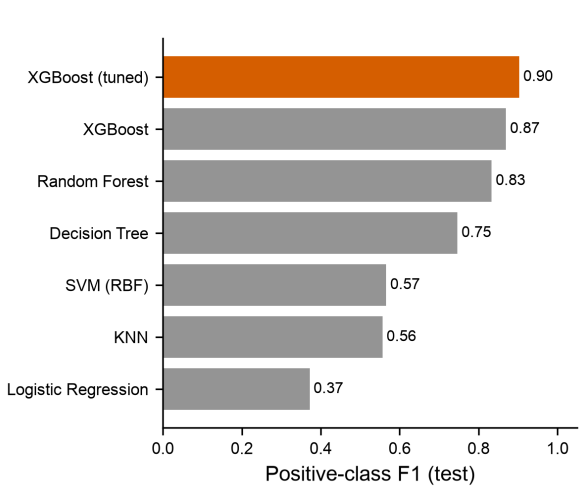
Table 4: Resampling technique comparison, validation set, fixed default XGBoost.

Sampler	Precision	Recall	F1	PR-AUC
Borderline-SMOTE	0.814	0.838	**0.826**	0.883
SMOTE	0.699	0.853	0.768	0.892
ADASYN	0.683	0.824	0.747	0.876

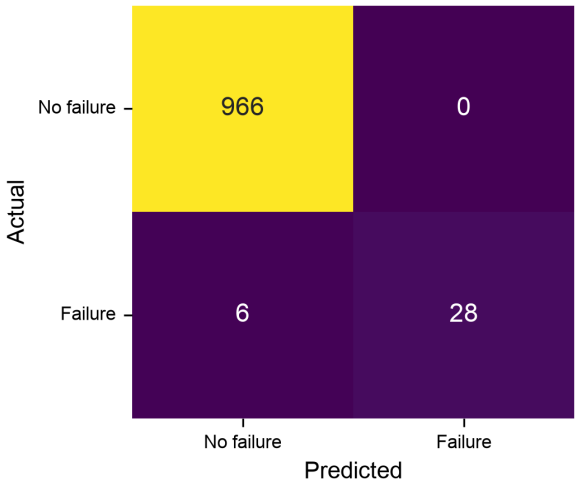
5.3 Dimensionality reduction

The engineered features are not redundant: Figure 3 shows the PCA explained-variance elbow, and 5 of the 12 processed features are needed to reach 95% cumulative variance, so there is little to compress away by design – consistent with each engineered feature capturing a distinct mechanism (Table 1) rather than a redundant transformation of the others.

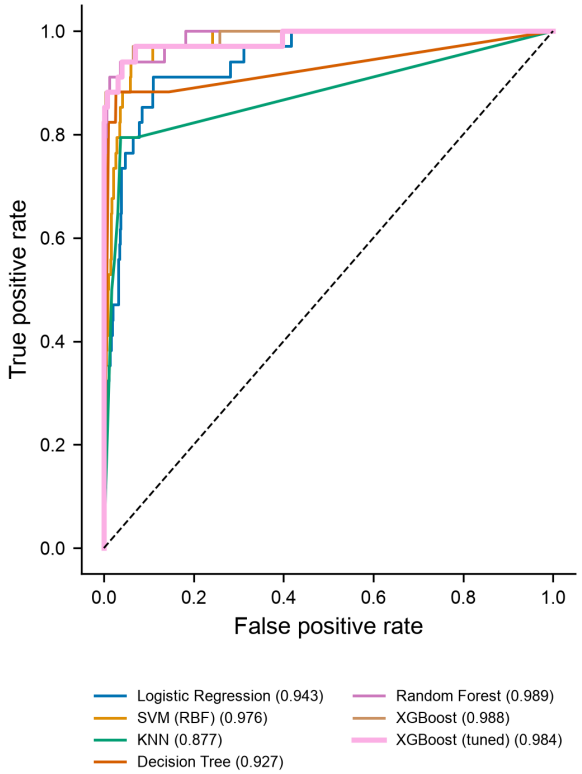
Figure 4 projects the training data into 2D with PCA, t-SNE [van der Maaten and Hinton, 2008], and UMAP [McInnes et al., 2018]. PCA (linear) shows the two classes smeared together with no separating direction, consistent with linear models' weak performance in Table 2. Both nonlinear methods instead show failure points clustering into several distinct pockets rather than one contiguous region – consistent with `Machine failure` being an aggregate of five mechanistically distinct causes rather than a single failure mode with one shared signature.



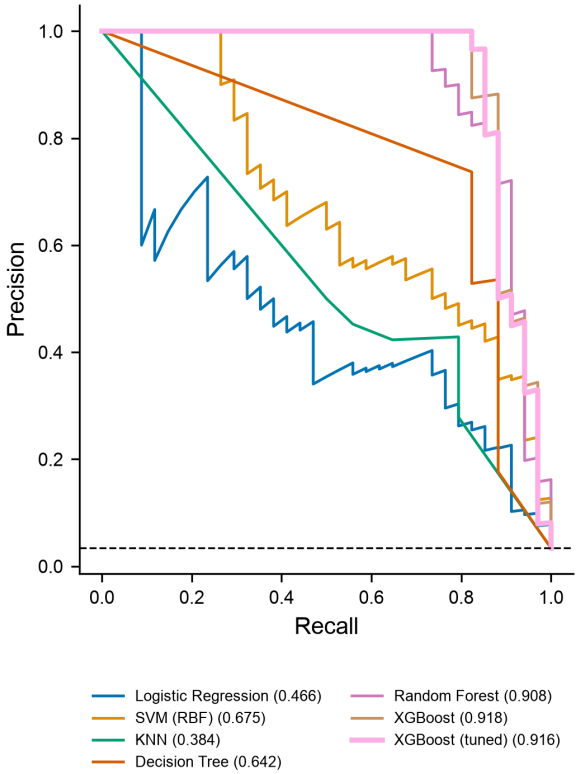
(a)



(b)



(c)



(d)

Figure 1: Test-set performance profile of the winning model. (a) Positive-class F1 across all classical models (tuned XGBoost highlighted). (b) Confusion matrix for the tuned model at its selected threshold. (c) ROC and (d) precision-recall curves for all classical models, tuned XGBoost highlighted. The PyTorch MLP is omitted from (a), (c), (d) for legibility – its numbers are in Table 2.

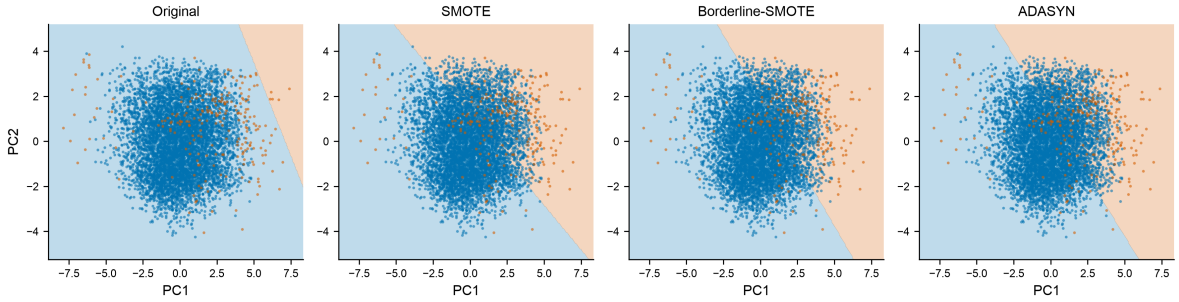


Figure 2: Logistic Regression decision boundary in 2D PCA space, by sampling technique. Without resampling the boundary is almost entirely majority-class; each oversampler shifts it toward the minority region.

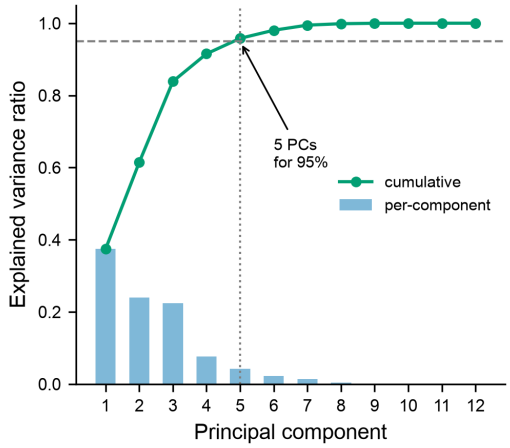


Figure 3: PCA explained-variance elbow on the processed training features. 5 of 12 components are needed for 95% cumulative variance.

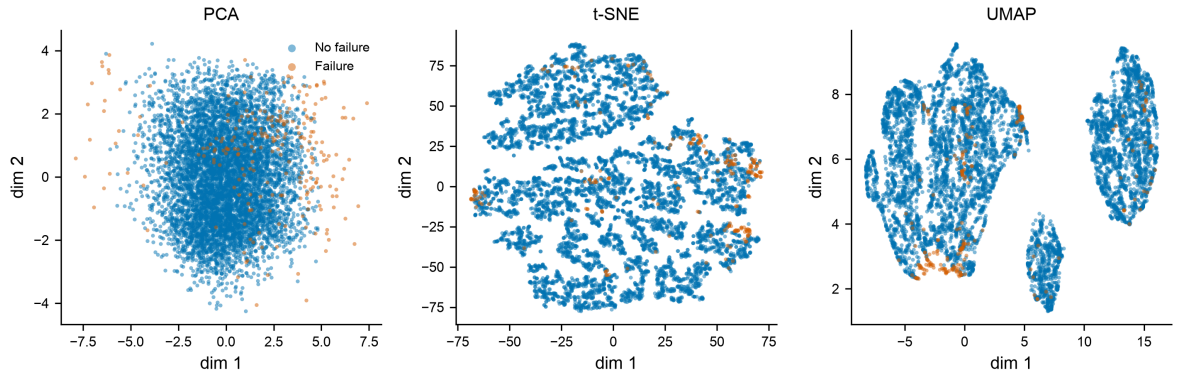


Figure 4: PCA vs. t-SNE vs. UMAP projections of the training data, colored by `Machine failure`.

t-SNE cluster shape and spacing can be an artifact of the perplexity parameter rather than a property of the data [Wattenberg et al., 2016], so Figure 5 reruns t-SNE on the same training data at perplexity $\in \{5, 30, 50, 100\}$. The failure points consistently fall into the same right-side region of the embedding across all four settings – the pocket structure in Figure 4 is not a one-off artifact of the perplexity=30 default used there.

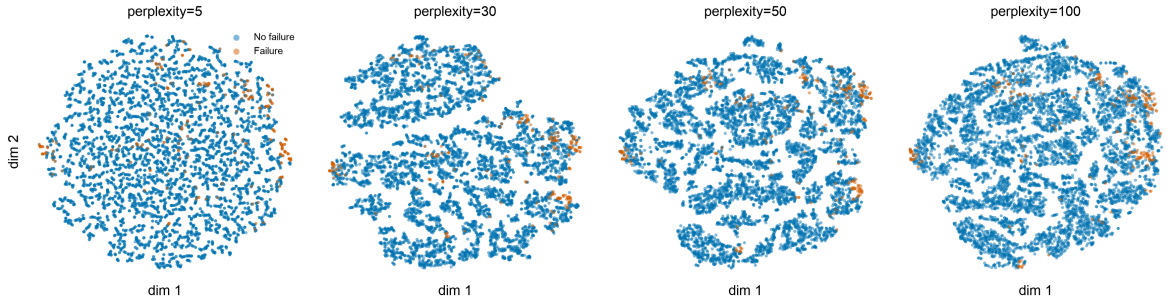


Figure 5: t-SNE embeddings of the training data at increasing perplexity. Failure points (orange) occupy a consistent region of the embedding at every setting.

5.4 XAI cross-check

Table 5 reports the protocol of Section 4.5 for the tuned model. All four physical mechanisms pass: on each mechanism's rows, the model concentrates attribution on that mechanism's causal features well beyond what any of the 100 permuted-label models does (e.g. PWF: $R = 0.686$ vs. a permuted-model mean of 0.331). The top attributed feature also matches the mechanism's driver in every case. Restricting to rows where exactly one mechanism fired leaves every verdict unchanged and slightly raises each R_m, so the result is not an artifact of overlapping mechanisms. RNF passes as well – the model is no more concentrated on RNF rows than a model that learned nothing – but, given that 18 of its 19 rows are non-failures (Section 3), we treat this as a weak check. Figure 6 shows the full per-feature, per-mechanism attribution underlying the table, and Figure 7 the global SHAP summary.

Table 5: Audit of the tuned model. R_m: share of mean |SHAP| on mechanism m's rows that falls on m's causal features (for RNF: share of the single top feature). Null: mean over 100 permuted-label models. p: model-randomization p-value (minimum 0.0099); "excl.": rows where only m fired. RNF passes if $p \geq 0.05$ (not more concentrated than the null).

Mech.	n	Top attributed feature	R_m	Null	p	p (excl.)	Result
TWF	46	Tool wear [min]	0.309	0.143	0.020	0.010	✓
HDF	115	Rotational speed [rpm]	0.705	0.553	0.010	0.010	✓
PWF	95	Power [W]	0.686	0.331	0.010	0.010	✓
OSF	98	Overstrain_ratio	0.664	0.344	0.010	0.010	✓
RNF	19	Rotational speed [rpm]	0.168	0.218	0.970	0.891	✓

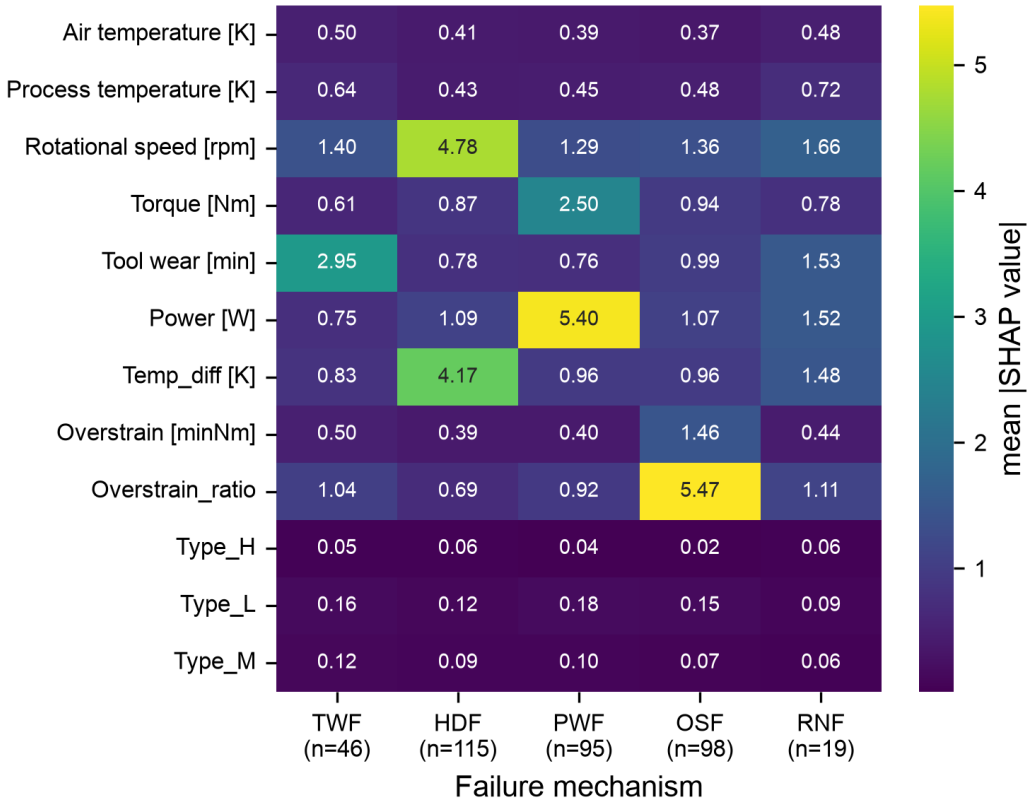


Figure 6: Mean |SHAP value| per feature, restricted to the rows where each mechanism fired.

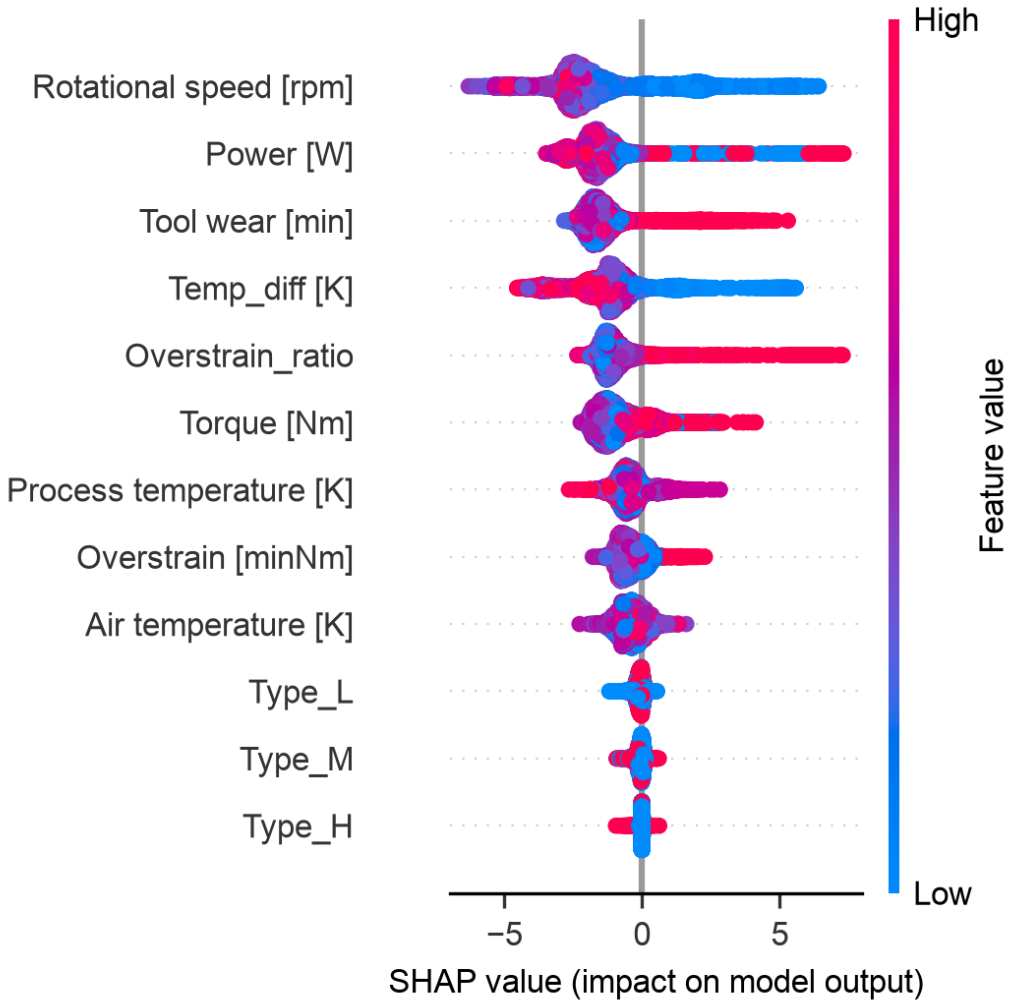


Figure 7: Global SHAP summary for the tuned model over the full dataset.

5.5 Can the audit fail? Controls and ablation

Table 6 runs the same audit, each with its own 100-model randomization null, on five variants of the pipeline.

Label permutation. A model trained on shuffled labels (test F1 0.08) fails all four physical mechanisms. The naive null from Section 4.5 passes this same model on all four ($p = 0.0005$ each), which is the concrete reason we reject it.

Spurious shortcut. We add a synthetic "sensor" with no physical role that tracks HDF, at two strengths. With a weak, noisy signal ($\mathcal{N}(0, 1) + 1.5\,\text{HDF}$) the model ignores it – the true HDF features already determine HDF exactly – and the audit correctly passes every mechanism. With a strong signal ($\mathcal{N}(0, 0.1) + \text{HDF}$) the model routes HDF through the shortcut: HDF's top feature becomes the fake sensor and R_{HDF} collapses from 0.705 to 0.156, a clear fail, while TWF, PWF and OSF still pass. The audit localizes the fault to the one mechanism the shortcut mimics. This model's test F1 is 0.86 – below the tuned model but above every non-XGBoost baseline in Table 2 – so accuracy alone would not have flagged it.

Raw features only. To address the concern that engineered features built from the trigger rules make the audit circular, we retrain without them. Test F1 drops from 0.90 to 0.84. TWF, PWF and OSF still pass clearly ($p = 0.0099$), with the expected raw drivers on top (torque for PWF, tool wear for OSF). HDF passes only marginally ($p = 0.0495$): its trigger is a *difference* of two temperatures, which a tree ensemble on raw inputs must approximate, and a know-nothing model already places much of its attribution on those same three features (null mean 0.60). The audit therefore has low power for difference-based mechanisms unless the relevant quantity is available as a feature.

Table 6: Audit p-values across pipeline variants (model-randomization null, $K = 100$; minimum $p = 0.0099$). Physical mechanisms pass if $p < 0.05$; $\times$ marks a fail. RNF passes if $p \geq 0.05$.

Variant	Test F1	TWF	HDF	PWF	OSF	RNF
Tuned model (engineered features)	0.90	0.020	0.010	0.010	0.010	0.970
exclusive-mode rows only	0.90	0.010	0.010	0.010	0.010	0.891
Raw features only	0.84	0.010	0.0495	0.010	0.010	0.376
Weak shortcut sensor	0.86	0.010	0.010	0.010	0.010	0.822
Strong shortcut sensor	0.86	0.010	1.000 $\times$	0.010	0.010	0.366
Permuted training labels	0.08	0.703 $\times$	0.535 $\times$	0.683 $\times$	0.366 $\times$	0.851

6 Discussion

6.1 What the controls show

The strong-shortcut model is the case the audit exists for: it scores competitively on held-out data, yet it is wrong about *why* one class of failure happens. Only the per-mechanism view exposes it, and it does so selectively – the other three mechanisms still pass. The permuted-label result makes a second, methodological point: an attribution audit whose null does not account for how atypical

the audited rows are will certify models that learned nothing. We expect this confound to apply to any subgroup-level attribution analysis, not only this one.

6.2 Limitations

AI4I 2020 is a synthetic dataset generated from the same closed-form rules used as our ground truth (Table 1); the "ground truth" is therefore the simulator's logic, not an independently verified physical measurement from real equipment. This is a genuine limitation for any claim that the protocol validates *real-world* causal faithfulness, and we do not make that claim here – our contribution is the protocol and its demonstration, not a validated causal finding about real machinery. This concern is not merely theoretical: Autran et al. [2024] show that injecting realistic irregularities (missing data, irregular time-series sampling) into AI4I 2020 measurably changes downstream model performance, underscoring that results on the clean synthetic version – ours included – may not transfer directly to messier real deployments.

Several further limitations are specific to the audit. The RNF check is weak: 18 of its 19 rows are non-failures, so it mainly shows the model does not concentrate attribution on arbitrary rows; the discriminating evidence comes from the permutation and shortcut controls. TWF (46 rows) is small, and its trigger is partly random, so its R is low in absolute terms even when it passes. With $K = 100$ permuted models the smallest attainable p is 0.0099, so several verdicts sit at that floor and more permutations would sharpen them. The audit has low power for mechanisms defined by a difference or ratio of raw inputs unless that quantity is a feature (HDF in the raw-feature ablation). We audit one model family (XGBoost) with one explainer (TreeSHAP); whether the null's behaviour carries over to other explainers is untested. Finally, the audit requires sub-labels for at least some failures – available here by construction, and in industrial settings where maintenance logs record a root cause, but not universal.

6.3 Generalizability

The protocol requires only two things: (1) a classifier trained on a coarse target, and (2) a held-out set of finer-grained sub-labels for at least some of the positive cases, not used in training. Both hold wherever maintenance records log a root-cause code for failures that a deployed model predicts only as a binary risk; we have not measured how widespread such records are. Where they exist, this protocol offers a cheap, dataset-native alternative to constructing synthetic ground-truth benchmarks purely to evaluate explanation faithfulness.

7 Conclusion

We show that AI4I 2020's five failure mechanisms, ordinarily discarded once the composite failure label is formed, can be repurposed as a faithfulness probe for post-hoc SHAP explanations. With a model-randomization null, a tuned XGBoost classifier trained only on the composite label passes all four physical mechanisms, a label-permuted model fails all four, and a model given a spurious shortcut fails exactly the mechanism the shortcut mimics while its accuracy stays competitive. Along the way we find that the obvious null for this kind of subgroup audit is confounded and certifies a model that learned nothing. We position the protocol as a low-cost check applicable wherever a coarse PdM target aggregates several distinct, partially-labeled failure modes – a complement to, not a replacement for, faithfulness evaluation on purpose-built synthetic data.

Acknowledgment

Portions of the manuscript text and the analysis code were drafted with the assistance of an AI system (Claude, Anthropic). The author reviewed and verified all content and takes full responsibility for it.

Code and data availability. The scripts in `experiments/` produce every number, table, and figure in this paper from the committed model artifacts; an exploratory notebook is included for context. Both are available at `https://github.com/kowshik24/glowing-broccoli`. AI4I 2020 is public [Matzka, 2020] and included in the repository.

References

Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

J.-V. Autran, V. Kuhn, J.-P. Diguet, M. Dubois, and C. Buche. AI4I-PMDI: Predictive maintenance datasets with complex industrial settings' irregularities. *Procedia Computer Science*, 246:1201–1209, 2024. doi: 10.1016/j.procs.2024.09.546.

Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

Shaunak Dhande, Chutian Ma, Giacinto Paolo Saggese, Paul Smith, and Krishna Taduri. A benchmark of causal vs. correlation AI for predictive maintenance. *arXiv preprint arXiv:2512.01149*, 2026.

Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. In *International Conference on Intelligent Computing*, pages 878–887. Springer, 2005.

Haibo He, Yang Bai, Edwardo A. Garcia, and Shutao Li. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks*, pages 1322–1328. IEEE, 2008.

Kushal Khemani and Anjum Nazir Qureshi. AI-driven predictive maintenance with environmental context integration for connected vehicles: Simulation, benchmarking, and field validation. *arXiv preprint arXiv:2603.13343*, 2026.

Yang Liu, Sujay Khandagale, Colin White, and Willie Neiswanger. Synthetic benchmarks for scientific research in explainable machine learning. *arXiv preprint arXiv:2106.12543*, 2021.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 2017.

Stephan Matzka. Explainable artificial intelligence for predictive maintenance applications. In *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*, pages 69–74. IEEE, 2020. doi: 10.1109/AI4I49448.2020.00023.

Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

Chun-Jan Tseng. A physics-informed benchmarking framework for machine learning and tree-based ensembles in IIoT-enabled predictive maintenance. *Sensors*, 26(16):5026, 2026. doi: 10.3390/s26165026.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

Simon Vollert and Andreas Theissler. Explainable predictive maintenance: A survey of current methods, challenges and opportunities. *arXiv preprint arXiv:2401.07871*, 2024.

Martin Wattenberg, Fernanda Viégas, and Ian Johnson. How to use t-SNE effectively, 2016. URL https://distill.pub/2016/misread-tsne/. Distill.

Wen Zhao, Jiawen Ding, Xueting Huang, and Yibo Zhang. Research on milling machine predictive maintenance based on machine learning and SHAP analysis in intelligent manufacturing environment. *arXiv preprint arXiv:2512.01205*, 2025.